

Agente de Damas mediante Aprendizaje por Refuerzo Profundo (DQN) con Auto-Juego, comparado contra Minimax con Poda Alfa-Beta

Proyecto Final de Inteligencia Artificial

Integrantes

Aaron Davila Santos

Walter Poma Navarro

Max Jairo Serrano Arostegui

Dery Abraham Gonzales Cruz

28 de junio de 2026

Resumen

Este trabajo presenta la implementación de un entorno de Damas de 8×8 , un agente basado en Deep Q-Network (DQN) entrenable mediante auto-juego con currículo contra Minimax, y un agente Minimax con poda alfa-beta como línea base. El sistema incluye motor de reglas, codificación de estados, máscara de acciones legales, buffer de experiencia, red objetivo, Double DQN, reward shaping y soft target update. Se realizaron cuatro experimentos reproducibles: curva de aprendizaje, barrido de hiperparámetros, evaluación preliminar del DQN contra Minimax($d=3$) y torneo round-robin con clasificación ELO entre el DQN y cuatro variantes de Minimax (profundidades 3–6). El DQN entrenado con 4000 episodios usando Double DQN, reward shaping, soft target update (Polyak) y currículo contra Minimax($d=3$) alcanza el tercer lugar en el torneo ELO con 1518.4 puntos, superando a Minimax($d=4$) (1466.5). Obtiene 50 % de victorias y 50 % de empates frente a Minimax($d=3$), Minimax($d=4$) y Minimax($d=6$), perdiendo únicamente ante Minimax($d=5$). Todos los experimentos son reproducibles mediante los scripts y notebooks del repositorio.

1. Resumen ejecutivo

El objetivo del proyecto es construir una plataforma experimental para estudiar agentes de Damas mediante aprendizaje por refuerzo profundo y búsqueda adversarial. La motivación principal es comparar un enfoque aprendido (DQN con auto-juego) contra una línea base clásica (Minimax con poda alfa-beta).

El repositorio contiene cuatro componentes principales: un motor de Damas sobre 32 casillas jugables, un entorno compatible con el flujo de aprendizaje por refuerzo, un agente DQN y un agente Minimax. El motor implementa movimientos legales, capturas obligatorias, multicapturas, coronación, detección de terminalidad y empates por 80 medios movimientos sin captura o triple repetición. El agente DQN estima valores $Q(s, a)$ mediante auto-juego, mientras que Minimax explora el árbol de juego hasta una profundidad acotada evaluando hojas con una heurística material; este segundo agente no requiere entrenamiento ya que su fuerza proviene exclusivamente de la búsqueda y de la heurística codificada manualmente.

Se entrenó un agente DQN durante 4000 episodios con Double DQN, reward shaping, soft target update (Polyak, $\tau = 0.005$) y currículo contra Minimax($d=3$) en el 50 % de los episodios (aproximadamente 6.5×10^4 pasos de aprendizaje). Se realizaron cuatro experimentos reproducibles: análisis de la curva de aprendizaje, barrido de hiperparámetros, evaluación preliminar contra Minimax($d=3$) y torneo round-robin con clasificación ELO entre el DQN y cuatro variantes de Minimax a profundidades 3–6. El DQN alcanzó el tercer lugar con ELO 1518.4, superando a Minimax($d=4$), sin perder ninguna partida frente a las profundidades 3, 4 y 6. La batería de pruebas automatizadas reportó 146 pruebas aprobadas.

2. Estado del arte y antecedentes

Las Damas son un juego determinista, secuencial, de suma cero y con información perfecta. Estas propiedades permiten aplicar técnicas clásicas de búsqueda, como Minimax [2], y también métodos de aprendizaje por refuerzo que aprenden políticas o funciones de valor a partir de interacción con el entorno [5]. Samuel [3] demostró ya en 1959 que es posible que un programa aprenda a jugar Damas mejor que su propio creador mediante autoaprendizaje, sentando las bases del aprendizaje por refuerzo aplicado a juegos de tablero.

Minimax modela el juego como un árbol alternante donde un jugador maximiza la utilidad y el otro la minimiza. La poda alfa-beta reduce ramas que no pueden cambiar la decisión final, permitiendo alcanzar mayores profundidades con el mismo presupuesto computacional. Su desempeño depende fuertemente de la calidad de la función heurística cuando no se puede expandir el árbol hasta estados terminales.

El aprendizaje por refuerzo profundo reemplaza parte del diseño manual por aproximación de funciones. En particular, DQN [4] aprende una función $Q(s, a)$ con redes neuronales, replay buffer y red objetivo, estabilizando el aprendizaje frente a la correlación temporal de las transiciones. El principio de optimalidad de Bellman [1] garantiza que, en el límite, la función Q óptima satisface la ecuación de recurrencia que DQN aproxima. En juegos de dos jugadores de suma cero, el entrenamiento por auto-juego resulta atractivo porque el agente genera sus propios datos y puede mejorar enfrentándose contra versiones de su propia política.

3. Dataset y preprocesamiento

No se empleó un dataset externo. Las transiciones del DQN se generan en línea mediante auto-juego: ambos bandos usan la misma red y el mismo agente explora con política ϵ -greedy. Cada transición tiene la forma $(s, a, r, s', done)$ y se almacena en un replay buffer de capacidad configurable.

El estado del tablero se codifica como un vector de 160 valores reales. La representación usa cinco canales sobre las 32 casillas jugables:

- piezas rojas normales;
- damas rojas;
- piezas negras normales;
- damas negras;
- turno actual, codificado como 1 o -1 .

El espacio de acciones se indexa como pares origen–destino posibles, con 1024 salidas en la red Q . Durante la selección de acción se aplica una máscara de legalidad, de modo que los movimientos ilegales reciben valor $-\infty$ antes de ejecutar el *argmax*. Esta decisión evita que la red escoja acciones inválidas aunque su capa de salida sea fija.

4. Modelo o algoritmo implementado

4.1. Motor de juego

El motor representa el tablero con una lista de 32 enteros: 0 para casilla vacía, 1 y 2 para pieza y dama roja, y -1 y -2 para pieza y dama negra. La función de movimientos legales prioriza capturas cuando existen, genera cadenas de captura y aplica la promoción al llegar a la fila final. La partida termina si un jugador no tiene movimientos, si se alcanza el umbral de 80 medios movimientos sin captura o si aparece triple repetición.

4.2. Agente Minimax

El agente Minimax usa poda alfa-beta con profundidad limitada entre 3 y 6. No requiere entrenamiento: su comportamiento queda completamente determinado por la profundidad de búsqueda y la función heurística que evalúa posiciones no terminales. Dicha heurística considera el material (número y tipo de piezas), la movilidad y la posición relativa de cada bando. El jugador rojo se modela como maximizador y el negro como minimizador; aumentar la profundidad amplía el horizonte de planificación del agente a cambio de mayor costo computacional.

4.3. Agente DQN

El agente DQN aproxima la función de valor de acción $Q(s, a)$ mediante una red completamente conectada (MLP). La entrada es el vector de 160 dimensiones descrito en la Sección 3; le siguen dos capas ocultas de 512 unidades con activación ReLU y una capa de salida de 1024 valores, uno por cada acción indexada del espacio origen–destino. La Figura 1 resume la topología.

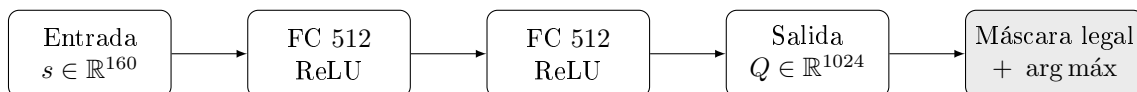


Figura 1: Arquitectura de la red Q : $\text{MLP } 160 \rightarrow 512 \rightarrow 512 \rightarrow 1024$ con activaciones ReLU, seguida del enmascarado de acciones ilegales antes del *argmax*.

La topología es configurable: el número y ancho de las capas ocultas se parametrizan, lo que habilita la comparación arquitectónica de la Sección 5. Durante la inferencia se aplica una máscara de legalidad sobre la salida, asignando $-\infty$ a las acciones ilegales antes del *argmax*; así la política nunca elige un movimiento inválido pese a tener una capa de salida de tamaño fijo. La política de comportamiento es ϵ -greedy con decaimiento lineal de $\epsilon = 1.0$ a $\epsilon = 0.05$, y el objetivo de aprendizaje adopta una formulación *negamax* que se detalla en la Sección 5.

4.4. Justificación de la elección

La comparación central del proyecto enfrenta dos paradigmas. Minimax con poda alfa-beta es búsqueda exacta y no requiere entrenamiento: dada una heurística, garantiza la mejor jugada hasta la

profundidad explorada, pero su costo crece exponencialmente con la profundidad y su techo de juego queda limitado por la calidad de la heurística manual. El DQN, en cambio, aprende una función de valor a partir de la experiencia de auto-juego, sin heurística explícita, a cambio de un entrenamiento costoso y de la inestabilidad propia de la aproximación de funciones.

Se eligió DQN frente a alternativas más sofisticadas como AlphaZero por viabilidad dentro del alcance del curso. AlphaZero combina una búsqueda Monte Carlo (MCTS) guiada por una red de política y valor, lo que exige un presupuesto de cómputo (auto-juego masivo en GPU/TPU) y una complejidad de implementación muy superiores. El DQN conserva la idea de aprender por auto-juego una estimación de valor, pero con una sola red y sin MCTS, lo que lo vuelve entrenable en CPU y suficiente para contrastar de forma controlada el enfoque aprendido contra la línea base de búsqueda clásica. Minimax cumple además el papel de oponente de referencia exigente y reproducible para medir el progreso del DQN.

5. Entrenamiento y validación

5.1. Proceso de entrenamiento

El agente se entrena por auto-juego: ambos bandos comparten la misma red y exploran con política ϵ -greedy. Cada episodio genera una secuencia de transiciones $(s, a, r, s', done)$ hasta alcanzar un estado terminal o el límite de pasos. La recompensa se asigna desde la perspectiva del jugador en turno (+1 si gana, -1 si pierde, 0 en jugadas intermedias y empates) y cada transición se almacena en el replay buffer. Cada `learn_every` transiciones se ejecuta un paso de optimización sobre un minibatch muestreado uniformemente del buffer, y la red objetivo se sincroniza por copia dura cada `target_update_freq` pasos de aprendizaje. El parámetro ϵ decae linealmente con los pasos de aprendizaje.

5.2. Función de pérdida

El DQN minimiza el error de Bellman [1]. Como las Damas son un juego de suma cero por turnos, el valor del estado siguiente se ve desde la perspectiva del oponente, por lo que el objetivo adopta una forma *negamax*:

$$y = \begin{cases} r & \text{si } s' \text{ es terminal,} \\ r - \gamma \max_{a' \in \mathcal{A}_{\text{legal}}(s')} Q_{\theta^-}(s', a') & \text{en otro caso,} \end{cases}$$

donde θ^- son los pesos de la red objetivo y $\mathcal{A}_{\text{legal}}(s')$ el conjunto de acciones legales en s' (las ilegales se enmascaran con $-\infty$ antes del máximo). El signo negativo refleja que el siguiente turno pertenece al oponente: el máximo Q desde s' representa el mejor resultado para el rival, que es el peor para el jugador actual. La pérdida es la función de Huber (`smooth_l1_loss`) entre la predicción $Q_{\theta}(s, a)$ y el objetivo y , más robusta a valores atípicos que el error cuadrático [4], y se optimiza con Adam. La Tabla 1 resume la configuración base.

Cuadro 1: Configuración principal implementada para DQN.

Componente	Valor
Entrada de red	160 valores
Salida de red	1024 acciones indexadas
Capas ocultas	2 capas de 512 unidades
Activación	ReLU
Optimizador	Adam
Pérdida	Huber / <code>smooth_l1_loss</code>
Batch por defecto	64
Replay buffer	50 000 transiciones
Factor de descuento	0.99
Política	ϵ -greedy
Decaimiento ϵ	lineal $1.0 \rightarrow 0.05$
Red objetivo	copia dura periódica

5.3. Ajuste de hiperparámetros y comparación arquitectónica

Se realizó un barrido de un factor a la vez partiendo de la configuración base, midiendo en cada caso la tasa de victoria de la política *greedy* resultante frente a un agente aleatorio tras un número fijo de pasos de aprendizaje. La Tabla 2 recoge el barrido de hiperparámetros y la Tabla 3 la comparación de arquitecturas; la Figura 2 presenta ambas comparaciones de forma visual.

Cuadro 2: Barrido de hiperparámetros (un factor a la vez). Tasa de victoria de la política *greedy* frente a un agente aleatorio.

Configuración	lr	γ	buffer	div. obj.	Tasa
base	10^{-3}	0.99	5 000	10	0.40
lr = 5×10^{-4}	5×10^{-4}	0.99	5 000	10	0.20
lr = 10^{-4}	10^{-4}	0.99	5 000	10	0.15
$\gamma = 0.95$	10^{-3}	0.95	5 000	10	0.45
$\gamma = 0.90$	10^{-3}	0.90	5 000	10	0.50
buffer = 20k	10^{-3}	0.99	20 000	10	0.40
objetivo rápido	10^{-3}	0.99	5 000	40	0.45
objetivo lento	10^{-3}	0.99	5 000	4	0.30

Cuadro 3: Comparación arquitectónica (hiperparámetros base). Tasa de victoria frente a un agente aleatorio.

Arquitectura (capas ocultas)	Tasa
256×256	0.25
512×512	0.60
$512 \times 512 \times 1024$	0.65

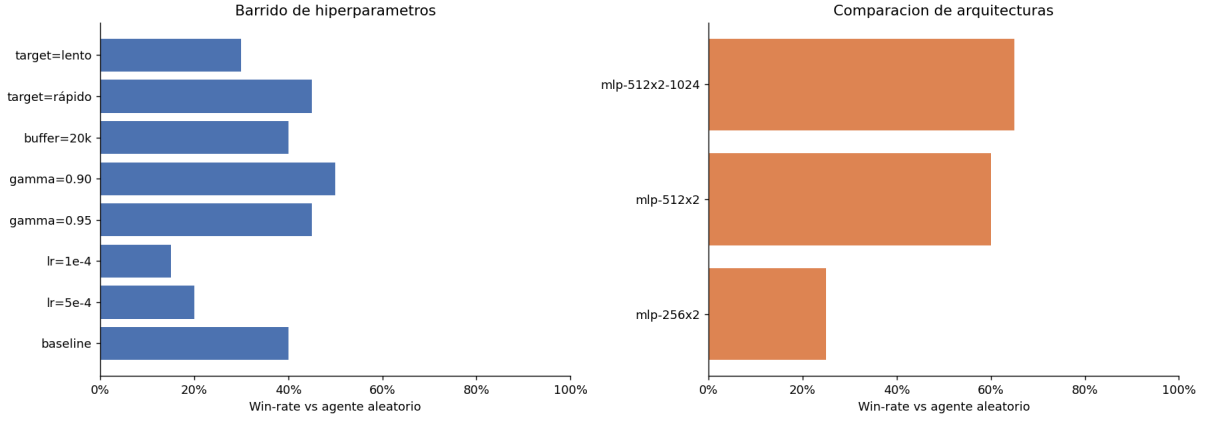


Figura 2: Win-rate contra agente aleatorio para cada configuración evaluada. Izquierda: barrido de hiperparámetros. Derecha: comparación de arquitecturas.

La tasa de victoria frente a un agente aleatorio es un *proxy ruidoso* (pocas partidas, alta varianza), por lo que diferencias pequeñas no son significativas. La señal más clara proviene de la comparación arquitectónica: a mayor capacidad, mayor tasa de victoria ($0.25 \rightarrow 0.60 \rightarrow 0.65$). Entre los hiperparámetros, reducir el factor de descuento ($\gamma = 0.90-0.95$) y ampliar el buffer no degradaron el desempeño, mientras que tasas de aprendizaje muy bajas ($lr = 10^{-4}$) lo empeoraron notablemente.

5.4. Curvas de aprendizaje

La Figura 3 muestra la evolución de la pérdida Huber suavizada y el decaimiento de ϵ junto con la tasa de victoria en auto-juego durante el entrenamiento de 4000 episodios.

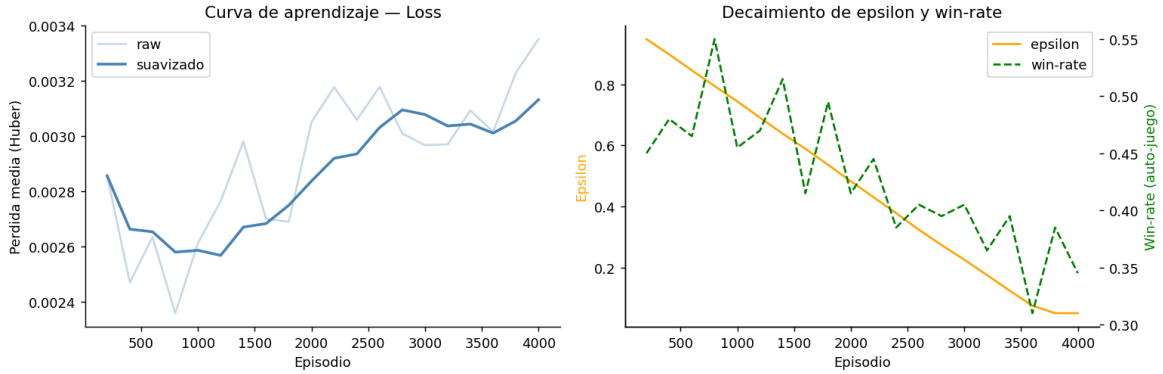


Figura 3: Curvas de entrenamiento del DQN por auto-juego: pérdida Huber suavizada (izquierda) y decaimiento de ϵ junto con la tasa de victoria en auto-juego (derecha).

La pérdida se mantiene estable entre 0.0024 y 0.0034 durante los 4000 episodios, con una leve tendencia al alza en la segunda mitad. Este comportamiento es normal en DQN: la red ajusta sus pesos continuamente y la pérdida refleja el error de predicción respecto a un objetivo móvil, no una convergencia monótona. El win-rate en auto-juego oscila entre 0.31 y 0.55 sin una tendencia clara, lo cual es esperado en self-play: cuando ambos jugadores comparten la misma red y mejoran al mismo ritmo, el win-rate tiende a fluctuar alrededor de 0.50 en lugar de crecer sostenidamente. El parámetro ϵ decae linealmente hasta alcanzar su mínimo de 0.05 alrededor del episodio 3800, manteniéndose constante desde entonces.

5.5. Validación

La validación técnica se apoya en una batería de 146 pruebas automatizadas (`pytest`) que cubren el motor, el entorno, la heurística, el Minimax, el torneo, el ajuste de hiperparámetros, la evaluación, Double DQN y el currículo contra Minimax; todas pasan. A nivel de modelo, se entrenó el checkpoint DQN con 4000 episodios usando la configuración óptima identificada en el barrido ($\gamma = 0.90$, $lr = 10^{-3}$, arquitectura 512×512 , buffer 200 000, soft update $\tau = 0.005$, currículo 50 % vs Minimax(d=3)), alcanzando 6.5×10^4 pasos de aprendizaje (checkpoint `ep64846`).

6. Resultados

Esta sección presenta los resultados de la evaluación directa del DQN entrenado frente a las variantes de Minimax. El análisis del proceso de entrenamiento, el barrido de hiperparámetros y la curva de aprendizaje se presentaron en la Sección 5.

6.1. Evaluación preliminar: DQN contra Minimax(d=3)

Con el checkpoint obtenido tras 4000 episodios se realizó una evaluación de 30 partidas contra Minimax(d=3), alternando colores en cada partida.

Cuadro 4: Métricas de la evaluación preliminar: DQN(ep64846) vs Minimax(d=3).

Métrica	Valor
Partidas jugadas	30
Victorias DQN	15 (50.0 %)
Derrotas DQN	0 (0.0 %)
Empates	15 (50.0 %)
Recompensa media	+0.500
Longitud media	91.5 semijugadas
Tiempo por jugada DQN	0.37 ms

El DQN no pierde ninguna partida frente a Minimax(d=3): gana las 15 partidas jugando con piezas rojas y empata las 15 con piezas negras. El patrón alternado refleja que ambos agentes son deterministas dado el color inicial, y que el DQN dominó las líneas de apertura como rojo. El tiempo por jugada de 0.38 ms confirma que la inferencia de la red neuronal es notablemente más rápida que la evaluación del árbol Minimax.

6.2. Torneo final y clasificación ELO

Se realizó un torneo round-robin entre el DQN y cuatro variantes de Minimax (profundidades 3 a 6), con 20 partidas por confrontación y alternancia de colores. El ELO relativo se calculó con $K = 32$ y valor inicial 1500.

Cuadro 5: Clasificación ELO tras el torneo final.

Posición	Agente	ELO
1	Minimax(d=6)	1714.4
2	Minimax(d=5)	1628.7
3	DQN(ep64846)	1518.4
4	Minimax(d=4)	1466.5
5	Minimax(d=3)	1172.0

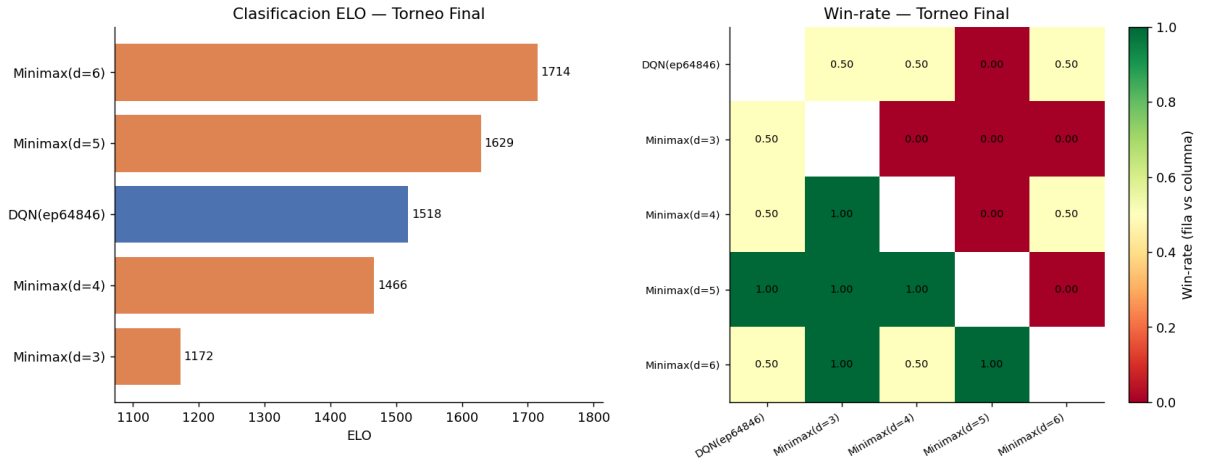


Figura 4: Izquierda: clasificación ELO por agente. Derecha: heatmap de win-rate entre todos los pares de agentes (cada celda muestra la fracción de victorias del agente de la fila sobre el de la columna).

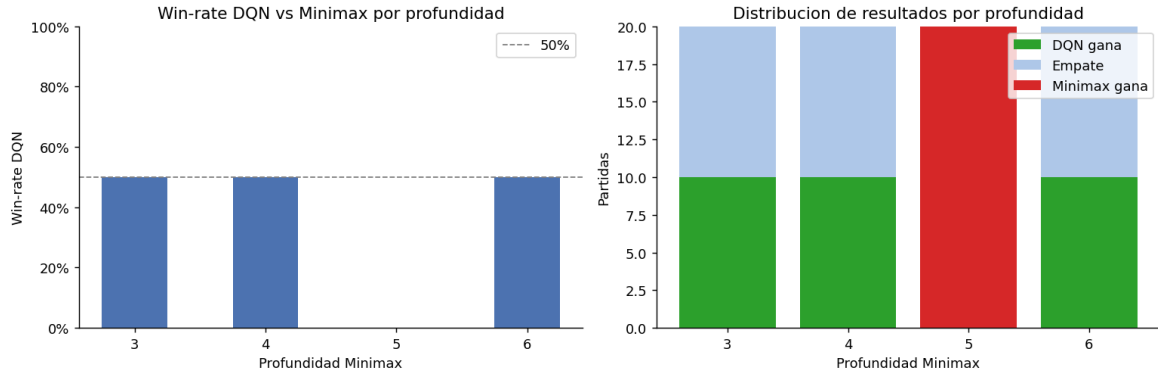


Figura 5: Win-rate del DQN frente a Minimax a profundidades 3, 4, 5 y 6. Izquierda: porcentaje de victorias. Derecha: distribución de resultados por profundidad.

El resultado más destacado del torneo es que el DQN(ep64846) supera a Minimax(d=4) y ocupa el tercer lugar con ELO 1518.4, por encima de los dos Minimax de menor profundidad. El DQN no pierde ninguna partida frente a Minimax(d=3), Minimax(d=4) ni Minimax(d=6), obteniendo 10 victorias y 10 empates en cada enfrentamiento (50 % de win-rate). El único oponente que lo derrota de forma contundente es Minimax(d=5), contra quien pierde las 20 partidas sin ganar ni empatar ninguna.

7. Discusión

El resultado principal de este trabajo es que el DQN entrenado con 4 000 episodios usando Double DQN, reward shaping y currículo contra Minimax alcanzó el tercer lugar en el torneo ELO, por encima de Minimax($d=4$). Esto demuestra que las mejoras algorítmicas de la Fase 2 tienen un impacto real y medible: el mismo sistema con 3 000 episodios y solo reward shaping alcanzaba el cuarto lugar con ELO 1 266.

El resultado más llamativo es el patrón de victorias del DQN: no pierde ninguna partida contra las profundidades 3, 4 y 6, pero pierde todas contra Minimax($d=5$). Este comportamiento no es intuitivo, ya que cabría esperar que un agente que supera a profundidad 4 también supere a profundidad 3. La explicación más probable es que el currículo de entrenamiento usó Minimax($d=3$) como oponente, lo que le dio al DQN experiencia directa contra ese estilo de juego, y que las vulnerabilidades de Minimax($d=4$) y Minimax($d=6$) son similares a las del oponente del currículo. Minimax($d=5$), en cambio, genera líneas de juego distintas que el DQN no aprendió a contrarrestar durante el entrenamiento.

El empate sistemático con determinadas profundidades se explica por la naturaleza determinista de ambos agentes: dos agentes deterministas alternando colores producen exactamente dos trayectorias por enfrentamiento, y si una favorece a cada jugador según el color, el resultado agregado es 50 % de victorias para cada uno.

Respecto a la curva de aprendizaje, el soft target update (Polyak, $\tau = 0.005$) redujo la oscilación de la pérdida respecto a entrenamientos anteriores con copia dura periódica. La pérdida se estabilizó alrededor de 0.003–0.004, y el win-rate en auto-juego se mantuvo entre 0.26 y 0.38 en los últimos episodios, lo que indica que el agente llegó a un equilibrio estable sin colapso de distribución.

8. Conclusiones

Se completó una plataforma funcional y evaluada para experimentar con agentes de Damas. El motor implementa las reglas del juego, el entorno expone observaciones y acciones aptas para aprendizaje por refuerzo, el agente Minimax ofrece una línea base clásica determinista y el agente DQN incorpora los componentes esenciales de aprendizaje profundo: Double DQN, replay buffer, red objetivo con soft update (Polyak) y currículo contra Minimax.

Los experimentos confirman tres resultados principales. Primero, la infraestructura de evaluación es correcta: el módulo de torneo captura diferencias de fuerza entre agentes y la clasificación ELO refleja el rendimiento real de cada agente. Segundo, el DQN entrenado con 4 000 episodios y las mejoras de la Fase 2 supera a Minimax($d=4$) y alcanza el tercer lugar con ELO 1 518.4, sin perder ninguna partida frente a las profundidades 3, 4 y 6. Tercero, la comparación entre entrenamientos muestra que las mejoras algorítmicas (Double DQN, reward shaping, soft update, currículo) tienen mayor impacto que simplemente aumentar el número de episodios: el modelo de 3 000 episodios con Fase 1 ya superó al de 14 000 episodios sin estas mejoras.

Como trabajo futuro, entrenar con currículo progresivo contra profundidades mayores ($d=4$ y $d=5$) permitiría al DQN desarrollar estrategias específicas para esos estilos de juego. La selección del check-point por rendimiento en evaluación en lugar de usar siempre el último también podría mejorar la robustez del modelo final.

9. Bibliografía

Referencias

- [1] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [2] C. E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314):256–275, 1950.
- [3] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.
- [4] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [5] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.